

LINUX COMMANDS – BEGINNER NOTES

1. pwd

Full Form: Print Working Directory

Purpose:

Shows the directory/location where you are currently working.

Syntax:

```
pwd
```

Example:

```
pwd
```

Output:

```
/home/maadhesh
```

Meaning:

You are currently inside `/home/maadhesh`.

Easy Memory:

`pwd` → Where am I?

2. ls

Purpose:

Lists the files and folders in the current directory.

Syntax:

```
ls
```

Example:

```
ls
```

Output:

Documents Downloads notes.txt Pictures

Easy Memory:

`ls` → List

3. `ls -l`

Purpose:

Displays files and folders with detailed information.

Syntax:

`ls -l`

Example Output:

```
-rw-r--r-- 1 maadhesh maadhesh 120 Sep 18 10:30 notes.txt
```

It can show:

- Permissions
- Owner
- Group
- File size
- Date/time
- File name

Easy Memory:

`-l` → Long / detailed

4. `ls -a`

Purpose:

Shows all files, including hidden files.

Linux hidden files usually start with `..`

Syntax:

ls -a

Example Output:

. .. .bashrc .profile notes.txt

Easy Memory:

-a → All

5. **ls -la**

Purpose:

Shows all files, including hidden files, with detailed information.

Syntax:

ls -la

Meaning:

-l → Detailed

-a → All

Easy Memory:

ls -la → All + Detailed

6. **ls -lh**

Purpose:

Shows detailed file information with file sizes in human-readable format.

Syntax:

ls -lh

Example:

-rw-r--r-- 1 maadhesh maadhesh 1.2K notes.txt

Instead of showing a large number of bytes, it may show K, M, G, etc.

Meaning:

-l → Detailed
-h → Human-readable

7. **ls -R**

Purpose:

Lists files and folders recursively, including contents of subdirectories.

Syntax:

ls -R

Example:

Documents:
notes.txt

Documents/project:
code.py

Easy Memory:

R → **Recursive**

8. **ls -lt**

Purpose:

Lists files in detailed format and sorts them by modification time.

Syntax:

ls -lt

Newer modified files generally appear first.

Meaning:

-l → Detailed
-t → Time

9. **ls -lS**

Purpose:

Lists files in detailed format and sorts them by file size.

Syntax:

ls -lS

Larger files generally appear first.

Meaning:

-l → Detailed

-S → Size

DIRECTORY COMMANDS

10. **cd**

Full Meaning: Change Directory

Purpose:

Moves from one directory to another.

Syntax:

cd directory_name

Example:

cd Documents

Now you enter the **Documents** folder.

Easy Memory:

cd → **Change Directory**

11. **cd ..**

Purpose:

Moves one level up to the parent directory.

Example:

Current location:

/home/maadhesh/Documents

Run:

cd ..

Now:

/home/maadhesh

Easy Memory:

.. → Go one level up

12. cd ~

Purpose:

Moves directly to your home directory.

Syntax:

cd ~

Example:

/home/maadhesh

Easy Memory:

~ → Home

USER COMMANDS

13. whoami

Purpose:

Shows the username of the currently logged-in user.

Syntax:

```
whoami
```

Example Output:

```
maadhesh
```

Easy Memory:

whoami → **Who am I?**

14. **su**

Full Form: Switch User

Purpose:

Switches from the current user to another user.

Syntax:

```
su username
```

Example:

```
su john
```

Linux may ask for John's password.

Easy Memory:

su → **Switch User**

15. **sudo**

Purpose:

Runs a command with elevated/administrator privileges.

Syntax:

sudo command

Example:

```
sudo mkdir test
```

Here, `mkdir test` is executed with elevated privileges.

Easy Memory:

`sudo` → Run with administrator privileges

FILE CREATION COMMANDS

16. `touch`

Purpose:

Creates an empty file.

Syntax:

```
touch filename
```

Example:

```
touch notes.txt
```

This creates:

```
notes.txt
```

Easy Memory:

`touch` → Create file

17. `mkdir`

Full Form: Make Directory

Purpose:

Creates a new directory/folder.

Syntax:

```
mkdir folder_name
```

Example:

```
mkdir Linux
```

Creates:

```
Linux/
```

Easy Memory:

mkdir → **Make Directory**

FILE VIEWING COMMANDS

18. **cat**

Purpose:

Displays the contents of a file.

Syntax:

```
cat filename
```

Example:

```
cat notes.txt
```

If the file contains:

```
Hello Linux
```

Output:

```
Hello Linux
```

Easy Memory:

cat → **View file content**

19. less

Purpose:

Reads a file page by page. Useful for large files.

Syntax:

less filename

Example:

less notes.txt

Press:

q

to exit.

Easy Memory:

less → Read large files page by page

20. head

Purpose:

Displays the beginning of a file.

Syntax:

head filename

Example:

head notes.txt

It normally shows the first several lines.

Easy Memory:

head → Beginning

21. tail

Purpose:

Displays the end of a file.

Syntax:

tail filename

Example:

tail notes.txt

It normally shows the last several lines.

Easy Memory:

tail → End

COPY / MOVE / DELETE COMMANDS

22. cp

Meaning: Copy

Purpose:

Copies a file or directory.

Syntax:

cp source destination

Example:

cp notes.txt backup.txt

Before:

notes.txt

After:

notes.txt

backup.txt

The original file remains.

Easy Memory:

cp → Copy

23. mv

Meaning: Move

Purpose:

Moves a file/directory or renames it.

Move Example

```
mv notes.txt Documents/
```

The file moves into Documents.

Rename Example

```
mv notes.txt linux_notes.txt
```

This changes:

notes.txt

to:

linux_notes.txt

Easy Memory:

mv → Move / Rename

24. rm

Meaning: Remove

Purpose:

Deletes a file.

Syntax:

```
rm filename
```

Example:

```
rm notes.txt
```

The file is deleted.

Important:

Command-line deletion usually does not use the normal recycle bin, so use `rm` carefully.

Easy Memory:

`rm` → Remove

25. `rmdir`

Meaning: Remove Directory

Purpose:

Deletes an empty directory.

Syntax:

```
rmdir folder_name
```

Example:

```
rmdir test
```

This works when `test` is empty.

Easy Memory:

`rmdir` → Remove Directory

WRITING INTO FILES

26. `echo`

Purpose:

Displays text on the terminal.

Example:

```
echo "Hello Linux"
```

Output:

Hello Linux

It can also be used to write text into files.

27. >

Purpose:

Writes output into a file and **overwrites existing content**.

Example:

```
echo "Hello Linux" > notes.txt
```

The file contains:

Hello Linux

If you run:

```
echo "Welcome" > notes.txt
```

The file becomes:

Welcome

The previous content is replaced.

Easy Memory:

> → Overwrite

28. >>

Purpose:

Adds/appends output to the end of a file without deleting existing content.

Example:

```
echo "Hello Linux" > notes.txt  
echo "I am learning Linux" >> notes.txt
```

File content:

```
Hello Linux  
I am learning Linux
```

Easy Memory:

>> → Append / Add

29. **cat > filename**

Purpose:

Creates or overwrites a file and allows you to enter multiple lines.

Example:

```
cat > notes.txt
```

Now type:

```
Hello Linux  
I am learning commands  
Linux is useful
```

Press:

Ctrl + D

to finish.

The content is saved in the file.

30. **cat >> filename**

Purpose:

Adds multiple lines to the end of an existing file.

Example:

```
cat >> notes.txt
```

Type:

This is another line

Then press:

Ctrl + D

The new line is appended to the existing content.

SEARCH COMMANDS

31. **find**

Purpose:

Searches for files and directories.

Basic Syntax:

```
find location -name "filename"
```

Example

```
find . -name "notes.txt"
```

Meaning:

find

↓

Search

.

↓

Current directory

-name

↓

Search by name

notes.txt

↓

Name to search

It searches the current directory and its subdirectories.

Easy Memory:

find → Find file/folder

32. find . -type f -name "notes.txt"

Purpose:

Searches only for files named `notes.txt`.

Example:

```
find . -type f -name "notes.txt"
```

Here:

-type f → File

Easy Memory:

f → File

33. find . -type d -name "Documents"

Purpose:

Searches only for directories named `Documents`.

Example:

```
find . -type d -name "Documents"
```

Here:

-type d → Directory

Easy Memory:

d → Directory

34. grep

Purpose:

Searches for specific text inside files.

Syntax:

```
grep "text" filename
```

Example:

```
grep "Linux" notes.txt
```

If the file contains:

Linux is an operating system.

Linux is open source.

Windows is also an operating system.

Output:

Linux is an operating system.

Linux is open source.

Important Difference

find → Finds files/folders

grep → Finds text inside files

35. grep -i

Purpose:

Searches text without considering uppercase/lowercase.

Example:

```
grep -i "linux" notes.txt
```

It can match:

Linux
linux
LINUX
LiNuX

Easy Memory:

-i → Ignore case

36. **grep -n**

Purpose:

Displays the line number where the matching text occurs.

Example:

```
grep -n "Linux" notes.txt
```

Output:

```
1:Linux is an operating system.  
2:Linux is open source.
```

Easy Memory:

-n → Number

37. **locate**

Purpose:

Quickly searches for file paths using a database.

Example:

```
locate notes.txt
```

It may return:

```
/home/maadhesh/notes.txt  
/home/maadhesh/Documents/notes.txt
```

Important:

find → Searches filesystem

locate → Searches database

The **locate** database may not immediately contain newly created files.

TEXT EDITORS

38. nano

Purpose:

Simple text editor for beginners.

Example:

nano notes.txt

You can type/edit the file.

Important shortcuts

Ctrl + O → Save

Enter → Confirm filename

Ctrl + X → Exit

39. vi

Purpose:

Command-line text editor.

Example:

vi notes.txt

To start typing:

i

This enters Insert Mode.

After typing:

Esc

Then:

:wq

Press Enter.

Meaning:

:w → Write/Save

q → Quit

40. vim

Purpose:

Vim is an improved version of the **vi** editor.

Example:

vim notes.txt

Basic flow:

vim notes.txt

↓

i

↓

Type content

↓

Esc

↓

:wq

↓

Enter

USER AND GROUP COMMANDS

41. id

Purpose:

Shows information about the current user, including UID, GID and groups.

Example:

```
id
```

Example output:

```
uid=1000(maadhesh) gid=1000(maadhesh) groups=1000(maadhesh)
```

Easy Memory:

id → User ID information

42. groups

Purpose:

Shows which groups a user belongs to.

Example:

```
groups
```

For another user:

```
groups john
```

43. useradd

Purpose:

Creates a new user.

Example:

```
sudo useradd john
```

Creates the user:

```
john
```

44. **passwd**

Purpose:

Creates or changes a user's password.

Example:

```
sudo passwd john
```

Linux will ask you to enter the password.

45. **userdel**

Purpose:

Deletes a user account.

Example:

```
sudo userdel john
```

46. **groupadd**

Purpose:

Creates a new group.

Example:

```
sudo groupadd developers
```

Creates:

```
developers
```

47. **groupdel**

Purpose:

Deletes a group.

Example:

```
sudo groupdel developers
```

48. **usermod -aG**

Purpose:

Adds an existing user to an additional group.

Example:

```
sudo usermod -aG developers john
```

Meaning:

usermod → Modify user

-aG → Add to supplementary group

developers → Group name

john → Username

Important:

-a means append, so existing supplementary group memberships are preserved.

Easy Memory:

-aG → Add user to Group

MANUAL / HELP COMMAND

49. **man**

Full Form: Manual

Purpose:

Shows the manual/documentation for a command.

Examples:

```
man ls
```

```
man mkdir
```

```
man find
```

man grep
man ssh

Inside the manual

Arrow keys → Move
Space → Next page
/word → Search
q → Quit

Easy Memory:

man → Manual

NETWORK COMMANDS

50. **ifconfig**

Full Form: Interface Configuration

Purpose:

Displays/configures network interface information.

Example:

ifconfig

It can show information such as:

- Network interface
- IP address
- MAC address
- Interface status
- Network statistics

Note:

On many modern Linux systems, **ifconfig** may not be installed by default.

51. **ip a**

Meaning: `ip address`

Purpose:

Displays network interfaces and their IP address information.

Example:

```
ip a
```

It can show:

- Network interfaces
- IP addresses
- Interface status
- Other network information

`ip a` is the modern commonly used alternative to `ifconfig`.

SSH COMMAND

52. `ssh`

Full Form: Secure Shell

Purpose:

Connects securely to another computer/server over a network using the command line.

Syntax:

```
ssh username@IP_address
```

Example:

```
ssh maadhesh@192.168.1.20
```

Meaning:

```
ssh
```

↓

Secure Shell

```
maadhesh
```

↓
Username

192.168.1.20

↓
Remote computer IP

SSH is commonly used to manage remote Linux servers.

IMPORTANT KEYBOARD COMMANDS / SHORTCUTS

53. **Ctrl + C**

Purpose:

Stops/interrupts a currently running foreground command.

Example:

ping google.com

Press:

Ctrl + C

to stop it.

54. **Ctrl + L**

Purpose:

Clears the visible terminal screen.

It does **not** delete command history.

55. **Ctrl + D**

Purpose:

Signals the end of input.

Commonly used after:

```
cat > notes.txt
```

or:

```
cat >> notes.txt
```

Example:

```
cat > notes.txt
```

Type your content

Ctrl + D